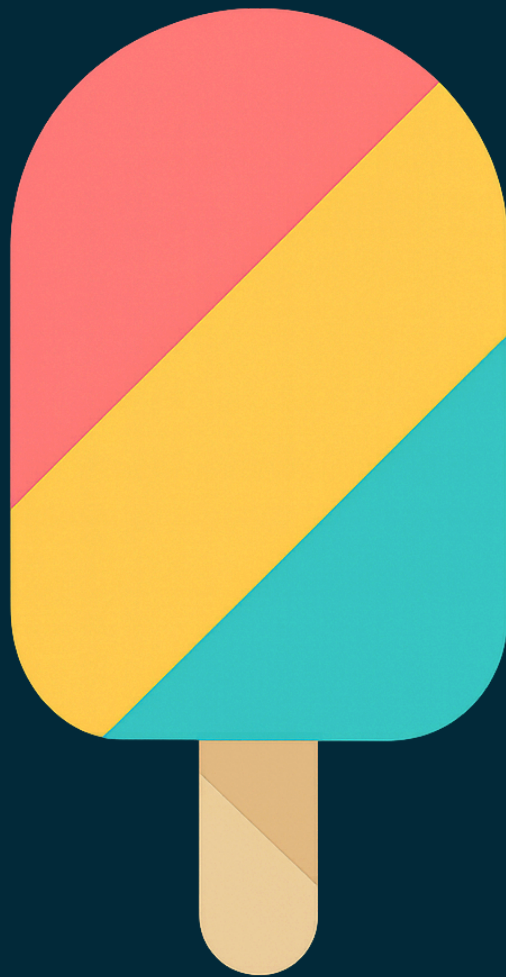


CNCF Project Focus

Episode #2



Crossplane

One Kubernetes API. Any Cloud Provider



Christian Dussol

The multi-cloud infrastructure challenge

Different platforms = Different tools

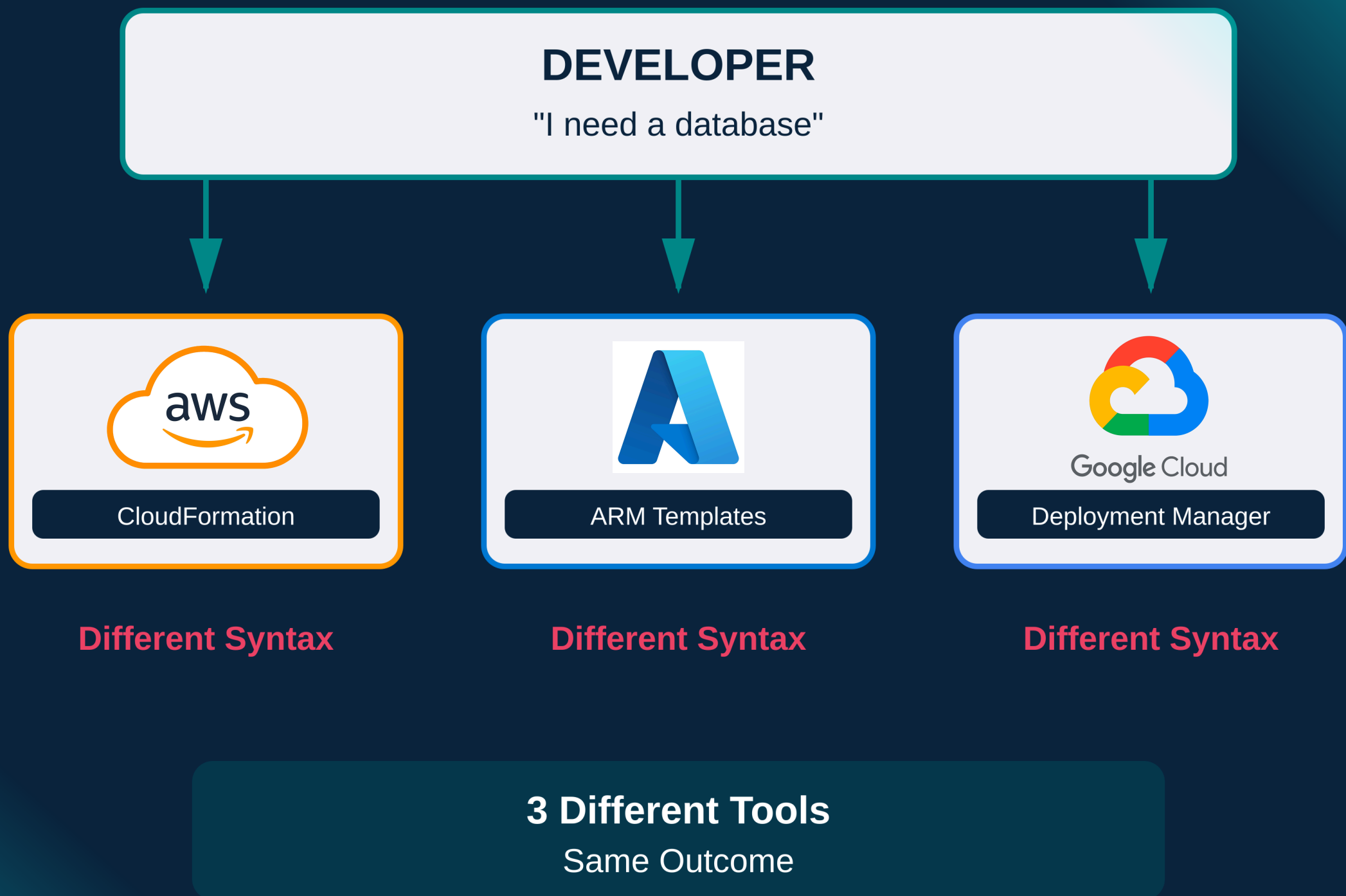
- **AWS:** CloudFormation + AWS CLI
- **Azure:** ARM Templates + Azure CLI
- **GCP:** Deployment Manager + gcloud
- **On-premise:** Ansible, scripts...

Impact

- Each platform = different syntax, different tooling
- No unified API for developers
- Infrastructure drift between environments
- Vendor lock-in at every layer

What if there was ONE consistent API for ALL platforms?

Multi-Cloud Fragmentation



Crossplane: one K8s API for all infra

CNCF Graduated project (December 2023)

Core Idea

Manage cloud infrastructure using Kubernetes API

Three Key Benefits

Unified Interface

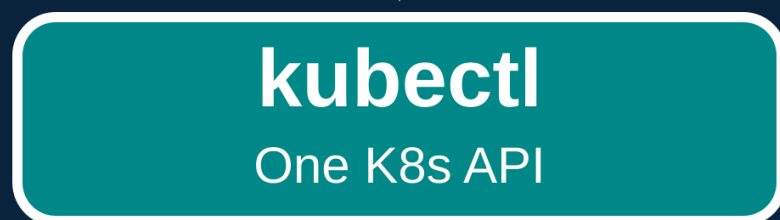
One API (kubectl) works with AWS, Azure, GCP and on-premise
No need to learn multiple tools

Cloud-Agnostic

Write infrastructure once, deploy anywhere
Switch providers without full rewrite

Customizable

Build your own infrastructure APIs
Match your organization's needs



Understanding the 4 core components



1. PROVIDER

Plugin that connects to a cloud platform
Example: provider-aws, provider-azure, provider-gcp

creates



2. MANAGED RESOURCE

Direct representation of a cloud resource
Example: RDS Instance, S3 Bucket, Virtual Machine

bundled into



3. COMPOSITION

Template that combines multiple resources
Example: Database + Security Group + Subnet + Backup

exposes via



4. XRD (Custom API)

Your simplified interface for developers
Example: Database API (hides cloud complexity)

From developer request to cloud resource

1

DEVELOPER REQUEST



"I need a production database with backups"

- Uses Custom API (XRD)
- Simple, abstract request

2

CROSSPLANE PROCESSING



- Finds matching Composition template
- Translates request → Managed Resources
- Creates: Database + SecurityGroup + Subnet + Backup
- Applies policies and governance

3

CLOUD PROVISIONING



- Provider talks to cloud platform (AWS/Azure/GCP)
- Creates actual infrastructure resources
- RDS instance, S3 buckets, networking configured
- Returns status to Crossplane

4

CONTINUOUS RECONCILIATION



- Crossplane monitors resource state continuously
- Detects configuration drift
- Auto-corrects to match desired state
- Self-healing infrastructure

Database Ready & Maintained

When to use Crossplane

GOOD FIT

1

Multi-Region Trading Platform

Deploy same infrastructure across
US (AWS), EU (Azure), APAC (GCP)

→ **One template, three clouds**

2

Developer Self-Service

50+ teams need databases with
encryption, backups, cost tags

→ **Standards enforced via API**

3

Cloud Migration (Hybrid)

Move from on-prem to cloud
gradually without disruption

→ **Unified API during transition**

4

Regulatory Compliance

Enforce encryption, backups,
audit trails automatically

→ **Policy-as-code with GitOps**

NOT FOR

Single Cloud Setup

Use CloudFormation, ARM, or
Terraform instead

No Kubernetes

Crossplane requires Kubernetes
Use Terraform for IaC

Small Team (<5 engineers)

Operational overhead too high
Keep it simple

Configuration Management

Use Ansible, Chef, or Puppet
Not infrastructure provisioning

Decision Guide



Choose Crossplane if: Multi-cloud + Kubernetes + Self-service needed



Skip Crossplane if: Simple setup + Single cloud + Small team

Explore my learning toolkit



github.com/christian-dussol-cloud-native/crossplane/

Educational GitHub Repository

What's Inside:

- ✓ **Quick start**
Command setup on local cluster
- ✓ **Multi-Cloud database example**
One template for AWS, Azure, and GCP
- ✓ **FinTech governance patterns**
Kyverno policies for backups and cost control
- ✓ **Cost Comparison Tool**
Calculate infrastructure costs across providers